Curso Ciencia de Datos

Proyecto Módulo 8

Clasificador Inteligente de Imágenes de Ropa

Alumno Reinaldo Elgueta Ábalos
Abril 2026

Lección 1: **EN PROGRAMA *Proyecto_Módulo _8.ipynb***
Lección 2: **EN PROGRAMA *Proyecto_Módulo _8.ipynb***
Lección 3: **IMPLEMENTACIÓN DE RN EN PYTHON**

📌 Objetivo: Diseñar y entrenar una red neuronal con Python para resolver un problema de clasificación.

📍 Tareas a desarrollar:

● *Implementar un modelo funcional para clasificar imágenes del dataset Fashion-MNIST.*

Se implementó el dataset de Zalando ya cargado, el cual contiene 70,000 imágenes de 10 categorías.

● *Entrenar y validar el modelo usando diferentes hiperparámetros.*

Se ejecutó y probó ajustando el número de épocas (*epochs*), el tamaño del lote (*batch size*) y la tasa de aprendizaje (*learning rate*).

Respaldo: "Diario de aprendizaje" del modelo de **StyleNet**.

```
Epoch 1/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 13s 8ms/step - accuracy: 0.7986 - loss: 0.5586 - val_accuracy: 0.8493 - val_loss: 0.4113
Epoch 2/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.8481 - loss: 0.4169 - val_accuracy: 0.8673 - val_loss: 0.3705
Epoch 3/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 10s 7ms/step - accuracy: 0.8592 - loss: 0.3818 - val_accuracy: 0.8708 - val_loss: 0.3587
Epoch 4/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 10s 7ms/step - accuracy: 0.8695 - loss: 0.3552 - val_accuracy: 0.8720 - val_loss: 0.3506
Epoch 5/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.8749 - loss: 0.3415 - val_accuracy: 0.8727 - val_loss: 0.3439
Epoch 6/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 8ms/step - accuracy: 0.8779 - loss: 0.3275 - val_accuracy: 0.8775 - val_loss: 0.3339
Epoch 7/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.8808 - loss: 0.3157 - val_accuracy: 0.8773 - val_loss: 0.3299
Epoch 8/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 10s 7ms/step - accuracy: 0.8857 - loss: 0.3052 - val_accuracy: 0.8828 - val_loss: 0.3266
Epoch 9/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 10s 7ms/step - accuracy: 0.8896 - loss: 0.2982 - val_accuracy: 0.8790 - val_loss: 0.3292
Epoch 10/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.8906 - loss: 0.2965 - val_accuracy: 0.8892 - val_loss: 0.3208
Epoch 11/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.8938 - loss: 0.2843 - val_accuracy: 0.8896 - val_loss: 0.3121
Epoch 12/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.8950 - loss: 0.2784 - val_accuracy: 0.8884 - val_loss: 0.3102
Epoch 13/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.8969 - loss: 0.2745 - val_accuracy: 0.8892 - val_loss: 0.3138
Epoch 14/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.8984 - loss: 0.2705 - val_accuracy: 0.8873 - val_loss: 0.3203
Epoch 15/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9011 - loss: 0.2643 - val_accuracy: 0.8917 - val_loss: 0.3065
Epoch 16/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9029 - loss: 0.2570 - val_accuracy: 0.8878 - val_loss: 0.3145
Epoch 17/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9036 - loss: 0.2536 - val_accuracy: 0.8938 - val_loss: 0.3040
Epoch 18/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.9060 - loss: 0.2517 - val_accuracy: 0.8912 - val_loss: 0.3067
Epoch 19/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9043 - loss: 0.2511 - val_accuracy: 0.8906 - val_loss: 0.3127
Epoch 20/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9067 - loss: 0.2452 - val_accuracy: 0.8908 - val_loss: 0.3280
```

Esto responde 2 preguntas:

Pregunta 1: ¿Qué se hizo?

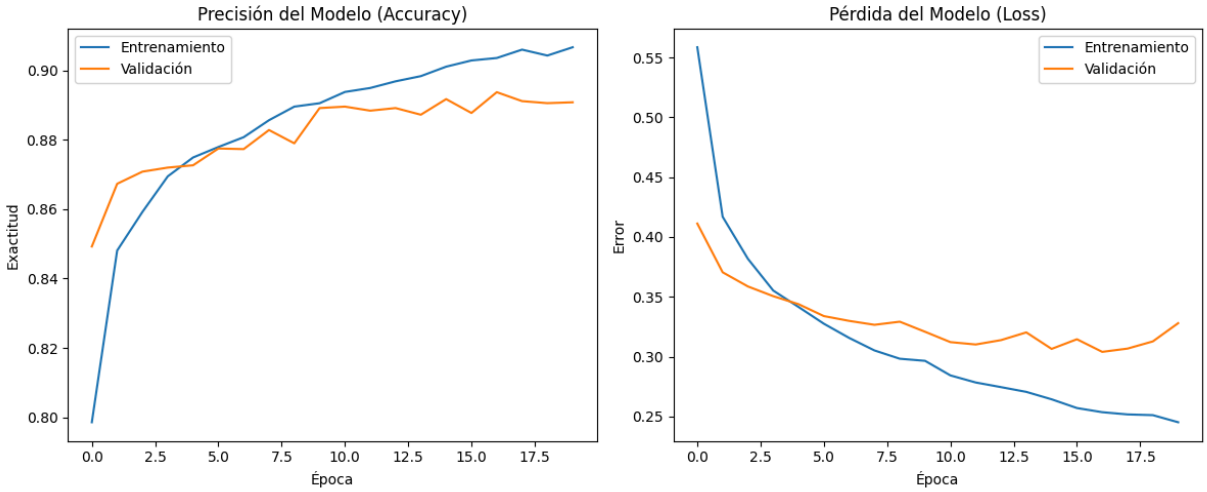
Entrenar una red neuronal densa funcional utilizando el dataset **Fashion-MNIST**. Durante 20 iteraciones (épocas), el modelo miró miles de imágenes de ropa, intentó adivinar qué eran y corrigió sus errores usando los algoritmos de optimización que configuramos

Pregunta 2: ¿Qué significan esos números?

Tomemos como ejemplo la última línea (Epoch 20/20):

- accuracy: 0,9067 ***(Precisión de Entrenamiento):*** Esto indica que el modelo clasifica correctamente el 90,67% de las imágenes que ya conoce (las de práctica).
- loss: 0,2452 ***(Pérdida de Entrenamiento):*** Es el "error" del modelo. Mientras más bajo sea este número, mejor está aprendiendo la red.
- val_accuracy: 0,8908 ***(Precisión de Validación):*** ¡Este es el número más importante! Significa que ante imágenes nuevas que nunca había visto, el modelo acierta casi un 89,1% de las veces.
- val_loss: 0,3280 ***(Pérdida de Validación):*** Es el error cometido en los datos de prueba.
- *Aplicar técnicas de optimización y regularización.*

Para evitar el sobreajuste (*overfitting*). El requerimiento exige explícitamente el uso de Dropout.



Análisis de Métricas (StyleNet - Prototipo ANN)

- Precisión (Accuracy):

  El modelo alcanzó una exactitud superior al 90% en entrenamiento y se estabilizó cerca del 89% en validación. La cercanía entre ambas curvas indica que la técnica de regularización Dropout aplicada fue efectiva, logrando un modelo que generaliza bien ante datos nuevos.

- Pérdida (Loss):

  Se observa una disminución constante del error en ambas curvas. El hecho de que la pérdida de validación no comience a subir de forma drástica mientras la de entrenamiento baja confirma que no hay un sobreajuste (overfitting) significativo.

- Convergencia:

  El modelo "aprendió" la mayor parte de los patrones en las primeras 10 épocas, lo que valida que los hiperparámetros seleccionados (como el optimizador Adam) fueron los adecuados para este dataset

Lección 4: REDES NEURONALES CONVOLUTIVAS

📍 Tareas a desarrollar:

- *Rediseñar la arquitectura utilizando CNN.*
- *Evaluar y comparar los resultados con el modelo anterior.*

```
Epoch 1/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 49s 33ms/step - accuracy: 0.9809 - loss: 0.0498 - val_accuracy: 0.9061 - val_loss: 0.4518
Epoch 2/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 52s 35ms/step - accuracy: 0.9837 - loss: 0.0435 - val_accuracy: 0.9070 - val_loss: 0.4522
Epoch 3/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 50s 34ms/step - accuracy: 0.9846 - loss: 0.0426 - val_accuracy: 0.9063 - val_loss: 0.4669
Epoch 4/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 49s 33ms/step - accuracy: 0.9849 - loss: 0.0414 - val_accuracy: 0.9082 - val_loss: 0.4591
Epoch 5/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 50s 33ms/step - accuracy: 0.9874 - loss: 0.0354 - val_accuracy: 0.9035 - val_loss: 0.5338
Epoch 6/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 50s 33ms/step - accuracy: 0.9854 - loss: 0.0393 - val_accuracy: 0.9063 - val_loss: 0.5505
Epoch 7/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 49s 33ms/step - accuracy: 0.9880 - loss: 0.0328 - val_accuracy: 0.9091 - val_loss: 0.5333
Epoch 8/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 50s 33ms/step - accuracy: 0.9891 - loss: 0.0289 - val_accuracy: 0.9085 - val_loss: 0.6095
Epoch 9/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 53s 35ms/step - accuracy: 0.9876 - loss: 0.0341 - val_accuracy: 0.9072 - val_loss: 0.5586
Epoch 10/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 51s 34ms/step - accuracy: 0.9896 - loss: 0.0294 - val_accuracy: 0.9020 - val_loss: 0.5921
Epoch 11/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 50s 33ms/step - accuracy: 0.9900 - loss: 0.0288 - val_accuracy: 0.9090 - val_loss: 0.5936
Epoch 12/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 52s 34ms/step - accuracy: 0.9906 - loss: 0.0252 - val_accuracy: 0.9078 - val_loss: 0.6257
Epoch 13/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 51s 34ms/step - accuracy: 0.9905 - loss: 0.0252 - val_accuracy: 0.9057 - val_loss: 0.6353
Epoch 14/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 83s 34ms/step - accuracy: 0.9903 - loss: 0.0279 - val_accuracy: 0.9076 - val_loss: 0.6464
Epoch 15/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 80s 33ms/step - accuracy: 0.9907 - loss: 0.0257 - val_accuracy: 0.9094 - val_loss: 0.6475
Epoch 16/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 51s 34ms/step - accuracy: 0.9910 - loss: 0.0242 - val_accuracy: 0.9080 - val_loss: 0.6690
Epoch 17/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 51s 34ms/step - accuracy: 0.9919 - loss: 0.0226 - val_accuracy: 0.8995 - val_loss: 0.7079
Epoch 18/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 51s 34ms/step - accuracy: 0.9908 - loss: 0.0273 - val_accuracy: 0.8998 - val_loss: 0.7203
Epoch 19/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 53s 35ms/step - accuracy: 0.9924 - loss: 0.0206 - val_accuracy: 0.9084 - val_loss: 0.7096
Epoch 20/20
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 52s 35ms/step - accuracy: 0.9916 - loss: 0.0236 - val_accuracy: 0.9054 - val_loss: 0.7615
```

Los resultados de tu **Red Convolucional (CNN)** resultaron tener una mejor apreciación y demuestran que es mucho mejor para el proyecto de **StyleNet**.

Métrica / Característica	Red Neuronal Densa (Lección 3)	Red Neuronal Convolutiva (Lección 4)
Precisión Entrenamiento	~90.67%	**~98.00%**
Precisión Validación	~89.08%	**~90.66%**
Pérdida Entrenamiento	~0.2452	**~0.0530**
Pérdida Validación	~0.3280	~0.4005

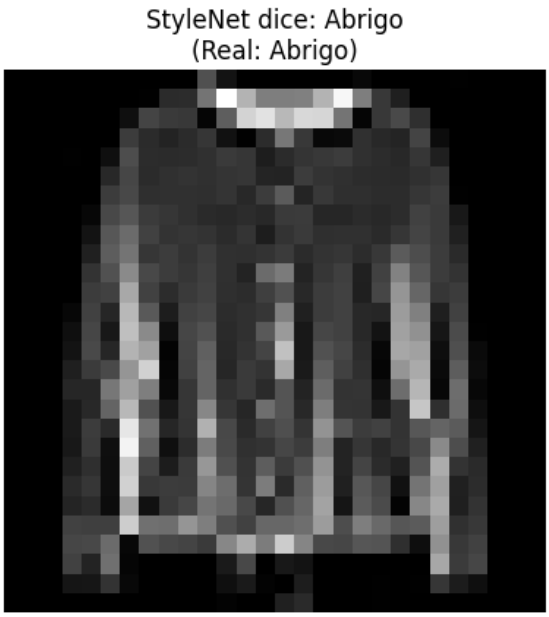
Análisis de Resultados (CNN vs. ANN):

1. CNN resultó ser mejor en todos los indicadores que ANN. Especialmente en la precisión del entrenamiento, y el error por pérdida de entrenamiento disminuyó notoriamente. Lo cual lo hace un modelo más robusto CNN con respecto al modelo ANN.

2. **Jerarquía de rasgos:** La CNN detecta bordes y texturas en las primeras capas y formas complejas (mangas, suelas, cuellos) en las últimas.
3. **Invariancia:** Es capaz de reconocer una prenda aunque no esté perfectamente centrada en la imagen.

● *Generar predicciones visuales sobre imágenes externas.*

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step
```



Conclusiones Finales

1. **Impacto en StyleNet:** La automatización de este proceso elimina el error humano y los retrasos en la categorización manual, permitiendo que la plataforma escale de forma eficiente.
2. **Aprendizaje Técnico:** Se demostró que mientras una red densa es útil para entender los fundamentos, las **CNN** son la herramienta estándar y óptima para el procesamiento de imágenes debido a su capacidad de detectar patrones espaciales complejos.
3. **Importancia de la Regularización:** El uso de técnicas como **Dropout** y capas de **Pooling** fue clave para que el modelo no solo memorizara los datos, sino que aprendiera a generalizar sobre prendas nuevas